גרסה 1 - מימוש ראשוני – 2026

גרסה 1 עוסקת בהקמה ובמימוש שכבות ה-domain, השירות (service) והתשתית (infrastructure) של מערכת המסחר. שכבת השירות משמשת להפעלת סיפורי השימוש. בסיום גרסה זו, הפעלת המערכת תהיה באמצעות מבחנים (tests) בלבד. גרסה 1 נבנית על בסיס עבודת המידול המקדימה שנעשתה בגרסה 0. בכל שלב **יש לשמור על תאימות בין המודלים והמימוש**.

כמו כן, שימו לב למדיניות השימוש במודלי שפה המופיעה באתר הקורס; אי ציות למדיניות עלולה להוביל להורדת ניקוד, ובמקרים חמורים אף להעלאה לוועדת משמעת.

תפקידי צוות: מנהל/ת גרסה ומפתחים.

הקלות לקבוצות עם היקף משמעותי של ימי מילואים וקבוצות של 7 סטודנטים מפורטות בתחתית המסמך.

הדרישות לגרסה 1:

המימוש של חלק מהדרישות מגרסה 0 נדחה לגרסאות עתידיות. הייצוג של הדרישות הללו במודלים צריך להישאר.

1. **דרישות רמת שירות**:
   a. עקביות ומניעת כפילויות (1) מתקיימת. בפרט, כל אילוצי הנכונות וכללי היושרה המפורטים במסמך הדרישות הכללי נשמרים לאחר כל פעולה. כמו כן, המימוש נדרש להתמודד עם race conditions הנובעים ממשתמשים הפועלים במערכת במקביל.
   b. דרישת הפרטיות והאבטחה (2): על המערכת לשמור סיסמאות של משתנים באופן מוצפן ולדאוג למנגנון מאובטח (מבוסס tokens) להתקשרות עם המערכת לאחר החיבור.
   c. דרישת קיבול עומס וזמינות (4): על המערכת לתמוך במספר משתמשים מכל הסוגים בו זמנית ועל המערכת להיות נגישה.
   d. דרישת המעקב (8): יש לתחזק יומן אירועים (event log) ויומן שגיאות (error log). היומנים תמיד זמינים לצפייה. יומן האירועים מתעד את הפניות למערכת (אילו תרחישים מופעלים עם אילו פרמטרים). יומן השגיאות מתעד שגיאות של המערכת. תרחישים שליליים אינם שגיאות.
2. **דרישות פונקציונאליות**:
   a. **מערכת:** דרישות I.1, I.2, I.3, I.4, I.7; בשלב זה אין חיבור למערכות חיצוניות אמיתיות, אך נדרש proxy לחיבור לצורך התממשקות עם ה-domain.
   b. **משתמשים:** מימוש כל הדרישות הפונקציונליות שפורטו בגרסה 0, מלבד מימוש דרישה II.4.3 - שינוי מדיניות קנייה של חנות. שימו לב שאינכם צריכים לתמוך בשלב זה בהתראות זמן אמת או התראות מושהות.

הנחיות למימוש:

מערכת בארכיטקטורה דו-שכבתית, בהתאם למודל הארכיטקטורה של המערכת.

1. **שכבת ה-domain**: לב המערכת בה ממומש המודל הלבן, והיא מחולקת לרכיבים אנכיים.
2. **שכבת האפליקציה (Application Layer)**: מנשק עבור סיפורי השימוש.
3. **שכבת התשתית (infrastructure layer):** המימוש הקונקרטי ל-repositories ולטכנולוגיות האחרות (מע' תשלומים \ כירטוס \ authentication וכו'...)
4. **תשתית בדיקות** ומימוש בדיקות קבלה ובדיקות פיתוח (יחידה ושילוב).
5. **GitHub repository**: הכולל ניהול קוד (כולל continuous integration) ופרוייקט.

דגשים למימוש:

1. **שימו לב לדגשים שניתנו במהלך הרצאת ההכנה לגרסה. דגשים אלו מהווים חלק בלתי נפרד מההנחיות.**
2. **מימוש המערכת**:
   a. יש לשים לב ששימוש לא זהיר במנעולים ובסינכרוניזציה יפגע בדרישת הזמינות.
   b. כדאי להוסיף ממשקים (interfaces) בכל מקום שבו יש סיכוי לשינוי מימוש בעתיד. כדאי גם להגדיר ממשק עבור אוספים של ישויות שממילא כבר יש להם מזהים.
3. **תחומי אחריות ואילוצי נכונות**: אילוצי נכונות צריך לאכוף באמצעות מבנה ופעולות המערכת, ולא להסתמך על בדיקות. יש לשים לב שפעולות מקביליות עלולות לגרום הפרה של אילוצי נכונות.
4. **עבודה עם Github Repository:**
   a. עליכם לנהל את הגירסאות של הקוד שלכם באמצעות github.
   b. משימות יש להגדיר ב-github project שלכם כאשר כל משימה משויכת ל-**milestone** של הגרסה הנוכחית.
   c. שימו לב - מצופה שה-commit האחרון שלכם (לפני יום הבדיקה וההגשה) יהיה משוייך ל-**tag** של הגרסה הנוכחית.
   d. לכל משימה יש לייצר issue branch ו-pull request המתאימים למשימה.
   e. יש לוודא שיש סימון מזהה של המשימה - bug, feature, …
   f. כל משימת פיתוח צריכה לכלול את ה-feature הנדרש ואת הבדיקות הרלוונטיות לאותו ה-feature. משימה שלא יוגדרו לה הבדיקות הרלוונטיות תוגדר כלא שלמה.
5. **יצירת CI - Continuous Integration:**
   מומלץ להיעזר בתוכן שנמצא בקישור: https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-java-with-maven
   a. יש לעבוד עם github actions על מנת לתמוך ב-continuous integration
   b. על ה-CI לכלול את כלל הבדיקות של כל התכולות שנוספו לגרסה.
   c. על ה-CI לרוץ כחלק מהשלבים שמתבצעים לפני שעושים push to main.
   d. ניתן להוסיף שלבים נוספים כראות עיניכם לפי הצורך.
6. **תשתית הבדיקות:**

a. יש לספק גם בדיקות **שליליות** (כאלו שכישלונן הוא הצלחתן), ו**בדיקות למקביליות**. כדי לכתוב בדיקות מקביליות עליכם תחילה לזהות תרחישי קצה של תחרות על משאבים (Race Conditions), לדוגמה שני משתמשים המנסים לרכוש את אותו הכרטיס בדיוק באותה אלפית שנייה. מבחינה טכנית, טסט רגיל אינו מספיק כאן; עליכם לייצר עומס מקבילי יזום על ידי הקמת מספר תהליכונים (Threads) הממתינים לנקודת זינוק אחידה, למשל באמצעות שימוש ב-ExecutorService ו-CountDownLatch ב-Java, כדי "להפגיז" את הפונקציה הרגישה בו-זמנית. בסוף הריצה, על הטסט לוודא את תקינות מצב המערכת. שימו לב שהטסטים צריכים לעבור ללא תלות בחירתכם להשתמש בנעילה אופטימית או פסימית.

b. הנחיות כלליות: **בדיקות קבלה ניגשות למערכת דרך שכבת האפליקציה בלבד**; יש להשתמש ב-Mocks (ע"י שימוש ב-mocking framework) במקרים המתאימים. תרחישי בדיקה צריכים להיות ממוקדים.

תוצרי גרסה 1:

מסמך המתודולוגיה מתאר את התוצרים השונים ואופן תיאורם. התוצרים בגרסה זו הם:

1. **מודלים עדכניים**
2. **מימוש המערכת**: לא יתקבל מימוש ללא רכיב בדיקות משמעותי!

הקלות למשרתי מילואים:

הקלות לקבוצות עם היקף שירות מילואים משמעותי.
שירות המילואים נקבע ע"פ מס' הימים המצטבר שחברי הקבוצה ביצעו החל מתחילת הסמסטר (15/3).
לצורך גרסה זו, יספרו הימים עד לתאריך 7/5 **כולל**. דוגמה: נניח שחבר קבוצה אחד שירת 10 ימי מילואים בטווח התאריכים, חברת קבוצה שניה שירתה 20, ויתר חברי הקבוצה לא שירתו, סך ימי המילואים הקבוצתי הינו 30.

ההקלות הינם בהתאם לשלוש מדרגות:

1. מדרגה א': סך מצטבר של 21+ ימים (או 0+ ימים בקבוצות של 7 סטודנטים).
2. מדרגה ב': סך מצטבר של 42+ ימים (או 21+ ימים בקבוצות של 7 סטודנטים).
3. מדרגה ג': סך מצטבר של 63+ ימים (או 42+ ימים בקבוצות של 7 סטודנטים).

הקלות למשרתים לפי מדרגות:

קבוצות **במדרגה א'** אינן נדרשות לממש את הדרישות הבאות:

- קבלת מידע ושינוי פרטים מזהים (II.3.4)
- הסרת מינוי בעל-חברה (II.4.9)
- ויתור על בעלות (II.4.10)
- השהיית/סגירת פעילות חברת ההפקה (II.4.13)
- פתיחת חברת הפקה שנסגרה (II.4.14)

קבוצות **במדרגה ב'** אינן נדרשות לממש את הדרישות הבאות:

- כל הדרישות של מדרגה א'
- ניהול עומסים (I.7)
- המתנה בתור (II.2.6)
- הפקת "דוח מכירות" (II.4.6)

קבוצות **במדרגה ג'** אינן נדרשות לממש את הדרישות הבאות:

- כל הדרישות של מדרגה א' ומדרגה ב'
- הרשמה להגרלת זכות רכישה (II.3.6)
- סגירת חברת הפקה (II.6.1)
- ביטול (הסרת) מנוי מהפלטפורמה (II.6.2)